Compute Jaccard distances on sparse matrix

Asked 7 years, 1 month ago Modified 3 years ago Viewed 8k times

8

I have a large sparse matrix - using sparse.csr_matrix from scipy. The values are binary. For each row, I need to compute the Jaccard distance to every row in the same matrix. What's the most efficient way to do this? Even for a 10.000 x 10.000 matrix, my runtime takes minutes to finish.

Current solution:

```
def jaccard(a, b):
    intersection = float(len(set(a) & set(b)))
    union = float(len(set(a) | set(b)))
    return 1.0 - (intersection/union)

def regions(csr, p, epsilon):
    neighbors = []
    for index in range(len(csr.indptr)-1):
        if jaccard(p, csr.indices[csr.indptr[index]:csr.indptr[index+1]]) <= epsilon:
            neighbors.append(index)
    return neighbors
csr = scipy.sparse.csr_matrix("file")
regions(csr, 0.51) #this is called for every row
```

python numpy scipy sparse-matrix Edit tags

Share Edit Follow Close Flag

Protect

edited Sep 28, 2015 at 8:13
user2379410

asked Sep 27, 2015 at 8:14

Kagemand Andersen
1,209 2 14 30

Could you elaborate on the details of the implementation that results in your runtime? – Ryan Sep 27, 2015 at 8:18

Ah,sorry. Code added. – Kagemand Andersen Sep 27, 2015 at 8:26

1 I would start by using a (presumably) optimized jaccard function, e.g. docs.scipy.org/doc/scipy/reference/generated/... – Ryan Sep 27, 2015 at 8:31

1 Or you can simply pass your entire array to `pairwise_distances` in sklearn using `'metric'='jaccard'` . Then you will probably benefit as well from optimized matrix operations being performed rather than looping. Also looks parallelizable (set `n_jobs=-1`) scikit-learn.org/stable/modules/generated/... You might need to do `csr.todense()` first. My guess is that you'll see a big increase in efficiency with this approach relative to your provided implementation – Ryan Sep 27, 2015 at 8:35

Tried it and it seems to take more time, and much more memory as the 'jaccard' metric doesn't support sparse. – Kagemand Andersen Sep 27, 2015 at 9:38

3 Answers

Sorted by:
Reset to default

Date modified (newest first)

-1

Here a solution that has a scikit-learn-like API.

```
def pairwise_sparse_jaccard_distance(X, Y=None):
    """
    Computes the Jaccard distance between two sparse matrices or between all pairs in
    one sparse matrix.

    Args:
        X (scipy.sparse.csr_matrix): A sparse matrix.
        Y (scipy.sparse.csr_matrix, optional): A sparse matrix.

    Returns:
        numpy.ndarray: A similarity matrix.
    """

    if Y is None:
        Y = X

    assert X.shape[1] == Y.shape[1]

    X = X.astype(bool).astype(int)
    Y = Y.astype(bool).astype(int)

    intersect = X.dot(Y.T)

    x_sum = X.sum(axis=1).A1
    y_sum = Y.sum(axis=1).A1
    xx, yy = np.meshgrid(x_sum, y_sum)
    union = ((xx + yy).T - intersect)

    return (1 - intersect / union).A
```

Here some testing and benchmarking:

```
>>> import timeit

>>> import numpy as np
>>> from scipy.sparse import csr_matrix
>>> from sklearn.metrics import pairwise_distances

>>> X = csr_matrix(np.random.choice(a=[False, True], size=(10000, 1000), p=[0.9, 0.1]))
>>> Y = csr_matrix(np.random.choice(a=[False, True], size=(1000, 1000), p=[0.9, 0.1]))
```

Asserting that all results are approximately equivalent

```
>>> custom_jaccard_distance = pairwise_sparse_jaccard_distance(X, Y)
>>> sklearn_jaccard_distance = pairwise_distances(X.todense(), Y.todense(), "jaccard")

>>> np.allclose(custom_jaccard_distance, sklearn_jaccard_distance)
True
```

Benchmarking runtime (from Jupyer Notebook)

```
>>> %timeit pairwise_jaccard_index(X, Y)
795 ms ± 58.3 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

>>> %timeit 1 - pairwise_distances(X.todense(), Y.todense(), "jaccard")
14.7 s ± 694 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

Share Edit Follow Delete Flag

answered Sep 30, 2019 at 14:58

Jan-Benedikt Jagusch
589 7 17

Extremely expensive on RAM (try a 20k row sparse matrix) – JALO - JusAnotherLivngOrganism Jun 11, 2020 at 18:57

1

To add to the accepted answer: I had use for a weighted version of the above method which is simply implemented as:

```
def pairwise_jaccard_sparse_weighted(csr, epsilon, weight):
    csr = scipy.sparse.csr_matrix(csr).astype(bool).astype(int)
    csr_w = csr * scipy.sparse.diags(weight)

    csr_rowsum = numpy.array(csr_w.sum(axis = 1)).flatten()
    intrsct = csr.dot(csr_w.T)

    rowsum_i = numpy.repeat(csr_rowsum, intrsct.getnnz(axis = 1))
    unions = rowsum_i + csr_rowsum[intrsct.indices] - intrsct.data
    dists = 1.0 - 1.0 * intrsct.data / unions

    mask = (dists > 0) & (dists <= epsilon)
    data = dists[mask]
    indices = intrsct.indices[mask]

    rownnz = numpy.add.reduceat(mask, intrsct.indptr[:-1])
    indptr = numpy.r_[0, numpy.cumsum(rownnz)]

    out = scipy.sparse.csr_matrix((data, indices, indptr), intrsct.shape)
    return out
```

I doubt this is the most efficient implementation, but it's a damn sight quicker than the dense implementation in `scipy.spatial.distance.jaccard`.

Share Edit Follow Flag

answered Oct 26, 2018 at 16:51

user5751
111 2

17

Vectorization is relatively easy if you use matrix multiplication to calculate the set intersections and then the rule `|union(a, b)| == |a| + |b| - |intersection(a, b)|` to determine the unions:

```
# Not actually necessary for sparse matrices, but it is for
# dense matrices and ndarrays, if X.dtype is integer.
from __future__ import division

def pairwise_jaccard(X):
```

```
    """Computes the Jaccard distance between the rows of `X`.
    """
    X = X.astype(bool).astype(int)

    intrsct = X.dot(X.T)
    row_sums = intrsct.diagonal()
    unions = row_sums[:,None] + row_sums - intrsct
    dist = 1.0 - intrsct / unions
    return dist
```

Note the cast to bool and then int, because the dtype of `x` must be large enough to accumulate twice the maximum row sum and that entries of `x` must be either zero or one. The downside of this code is that it's heavy on RAM, because `unions` and `dists` are dense matrices.

If you're only interested in distances smaller than some cut-off `epsilon`, the code can be tuned for sparse matrices:

```
from scipy.sparse import csr_matrix

def pairwise_jaccard_sparse(csr, epsilon):
    """Computes the Jaccard distance between the rows of `csr`,
    smaller than the cut-off distance `epsilon`.
    """
    assert(0 < epsilon < 1)
    csr = csr_matrix(csr).astype(bool).astype(int)

    csr_rownnz = csr.getnnz(axis=1)
    intrsct = csr.dot(csr.T)

    nnz_i = np.repeat(csr_rownnz, intrsct.getnnz(axis=1))
    unions = nnz_i + csr_rownnz[intrsct.indices] - intrsct.data
    dists = 1.0 - intrsct.data / unions

    mask = (dists > 0) & (dists <= epsilon)
    data = dists[mask]
    indices = intrsct.indices[mask]

    rownnz = np.add.reduceat(mask, intrsct.indptr[:-1])
    indptr = np.r_[0, np.cumsum(rownnz)]

    out = csr_matrix((data, indices, indptr), intrsct.shape)
    return out
```

If this still takes to much RAM you could try to vectorize over one dimension and Python-loop over the other.

Share Edit Follow Flag

answered Oct 1, 2015 at 11:01

user2379410

For the sparse approach since you are casting the CSR matrix as int, when you do the divisions you will only get either 0 or 1 as possible scores. I guess you could just cast it as float instead. – Laggel Oct 5, 2017 at 8:51

Brilliant, although I had to cast intrsct.data as a float for the division to go through properly. Whyever isn't this implemented in sklearn.spatial.distance? – user5751 Oct 25, 2018 at 17:26

Note that this returns a NumPy `matrix`, which is a strange and surprising data type if you're used to `ndarray`s. I changed the final line to `return dist.A` to make it return a conventional array. – Ken Arnold Jun 18, 2019 at 19:41